Algorithmics · P1 to P6

These programs are examples of courses. To understand them, specify entries, output and the assumptions, then execute a few steps by hand. The responses of the activities and DMs remain to be drafted separately.

P1 · Terms, sum and threshold

We assume. n positive or null integer and positive integer safety ceiling. The first function send back u_n and $\sum_{k=0}^{n} u_k$ for $u_{k+1} = au_k + b$. The second is looking for an increasing first crossing; its safety ceiling only indicates that it was not found in the number of steps set.

```
def term_sum(n, u0, a, b):
    u, total = u0, u0
    for k in rows (n):
        u = a*u + b
        Total += u
    return u, total

def Threshold(u0, a, b, target, maximum=100000):
    u, n = u0, 0
    while u <= target and n < maximum:
        u = a*u + b
        n += 1
    if u <= target:
        return None
    return n
```

P2 · Dichotomy

The calculations below use floating numbers; mathematical guarantee assumes signs Assumptions: f continues on $[a, b]$ and $f(a)f(b) \leq 0$. To identify a unique zero, separate proof of uniqueness. The function returns a length interval at the most ε, as long as machine precision still allows to share the interval.

```
def dichotomy(f, a, b, epsilon):
    assert a < b and epsilon > 0
    f, fb = f(a), f(b)
    assert fa*fb <= 0
    if Fa=0: return a, a
    if fb = 0: return b, b
    while b-a > epsilon:
        m = (a+b)/2
        if m == a gold m == b:
            raise ArithmeticError( "precision machine reached" )
        fm = f(m)
        if fm = 0: return m, m
        if f*fm < 0:
            b, fb = m, fm
        Else :
            a, fa = m, fm
    return a, b
```

P3 · Euler

With N strictly positive and $t_0 <$ end, for $y' = F(t,y)$, $Y_{k+1} = Y_k + hF(t_k, Y_k)$. The programme returns all approximations; it does not prove their accuracy.

```
def euler(F, t0, y0, fin, N):
    assert N > 0 and end > t0
    h = (end-t0)/N
    y, values = y0, [(t0, y0)]
    for k in rows (N):
        t = t0+k*h
        y = y+h*F(t, y)
        values.append((t0+(k+1)*h, y))
    return values
```

P4 · Rectangles

For N strictly positive and f continued and increasing on $[a,b]$, the result gives a limit Without monotonicity, these are only two approximations to be considered.

```
def rectangles(f, a, b, N):
    assert N > 0 and a < b
    h = (b-a)/N
    left = h* sum(f(a+k*h) for k in rows (N))
    right = h* sum(f(a+k*h) for k in rows (1, N+1))
    return left, right
```

P5 · Binomial distribution and simulation

We assume. n positive integer or zero, $0 \leq p \leq 1$ and k Integer. The direct calculation below is suitable for small sizes used in exercises; for very large parameters, libraries specialized digitals are preferable.

```
From math import comb
From random import Random

def likelihood(n, p, k):
    if k < 0 gold k > n: return 0.0
    return comb(n, k)*p**k*(1-p)**(n-k)

def cumulation(n, p, k):
    return sum (probability(n, p, j)
                for j in rows (min(k, n)+1))

def simulate_binomial(n, p, generator):
    return sum (generator.random() < p for _ in rows (n))

rng = Random(2026)
```

P6 · Fluctuations of frequency

We assume. N Total Strictly positive and $0 \leq p \leq 1$. The seed reproduces the same example. Repeat with other seeds; the law of large numbers does not say that the observed deviation decreases at each step.

```
From random import Random

def frequency(N, p, seed=2026):
    rng = Random(grain)
    success, result = 0, []
    for n in rows (1, N+1):
        success += rng.random() < p
        result.append(success/n)
    return result
```